Validating Transformer-Based Financial Encoders on Structured XBRL Data: A Benchmark Study

Elroy Galbraith
elroy.galbraith@gmail.com

Working Draft — March 27, 2026

Abstract

We evaluate the FT-Transformer architecture as a standalone encoder for structured financial data extracted from SEC EDGAR XBRL filings, benchmarking it against logistic regression, XGBoost, and gradient-boosted trees on multiple financial distress prediction tasks. We additionally investigate whether self-supervised pretraining via masked feature reconstruction improves downstream discriminative performance. This study serves as a validation gate for a larger research program investigating cross-modal joint-embedding predictive architectures for financial risk signals. All data extraction pipelines, feature definitions, and training code are released as open-source software.

[TODO: Revise abstract after results are finalized.]

1 Introduction

Deep learning has achieved strong results across many domains, yet its adoption for tabular financial data remains limited. Tree-based ensembles—particularly XGBoost (**?**) and gradient-boosted trees—continue to dominate applied financial modeling, and recent comprehensive benchmarks confirm that they remain competitive with or superior to deep learning on many tabular tasks (**??**). This presents a problem for research programs that aim to build multimodal financial architectures: if the base encoder for structured financial data cannot match conventional baselines, downstream gains from cross-modal integration are difficult to attribute.

The FT-Transformer (**?**) addresses several known failure modes of deep learning on tabular data. By tokenizing each input feature through a learned linear projection and processing the resulting sequence with standard Transformer attention, it avoids the feature-interaction bottlenecks of MLPs while preserving the inductive biases that make attention effective for heterogeneous input modalities. **?** demonstrated that the FT-Transformer matches or exceeds gradient-boosted decision trees on a range of tabular benchmarks, making it a plausible backbone for financial data.

However, financial tabular data presents distinct challenges that generic tabular benchmarks do not capture. XBRL (eXtensible Business Reporting Language) filings contain features with extreme tail distributions, systematic missingness driven by reporting standards rather than randomness, and temporal dependence structures that violate the i.i.d. assumptions of standard cross-validation. Whether the FT-Transformer's advantages generalize to this setting is an open empirical question.

Self-supervised pretraining offers an additional potential advantage. In the natural language and computer vision domains, pretraining on unlabeled data before fine-tuning on task-specific labels consistently improves performance, especially in low-label regimes (**??**). For financial data, where distress events are rare and ground-truth labels are expensive to obtain, even modest gains

from self-supervised initialization could be practically significant. We investigate masked feature reconstruction—a tabular analog of masked language modeling—as a pretraining strategy for the FT-Transformer.

This paper makes the following contributions:

1. We construct a reproducible dataset of 16 XBRL-derived financial features for the full universe of SEC 10-K filers from 2012 to 2024, with four binary distress outcome labels, using only publicly available data sources.

2. We benchmark the FT-Transformer against logistic regression (with both traditional ratios and full feature sets), XGBoost, and a histogram-based gradient-boosted tree on distress prediction, using a strictly temporal evaluation protocol that prevents look-ahead bias.

3. We evaluate masked feature reconstruction as a self-supervised pretraining objective for the FT-Transformer and measure its effect on downstream classification performance.

4. We release the complete data extraction pipeline, feature engineering code, model implementations, and evaluation scripts as open-source software to support reproducibility and further research.

This work constitutes Study 0 of the Fin-JEPA research program, a gated sequence of studies that progressively builds toward a cross-modal joint-embedding predictive architecture for financial risk signals (**?**). The results here determine whether the FT-Transformer is a viable base encoder for the structured financial modality before committing to cross-modal integration in subsequent studies.

2 Data

2.1 Company Universe

We construct a survivorship-bias-free universe of all entities that filed SEC Form 10-K annual reports between fiscal years 2012 and 2024. The universe is derived from two EDGAR data sources: (i) the quarterly full-index files, which enumerate all filings by type and date, and (ii) the per-company submissions API,[1] which provides metadata including ticker symbols, exchange listings, SIC codes, and fiscal year end dates.

The 2012 start date is chosen because XBRL tagging became mandatory for all SEC filers in that year, ensuring consistent structured data availability across the universe. The universe includes all entities regardless of current listing status—delisted, bankrupt, and acquired companies are retained to avoid survivorship bias. We assign each company to one of 12 sectors using the Fama–French industry classification mapped from 4-digit SIC codes.

The resulting universe contains approximately 8,000 unique companies. Each company-year observation is identified by its CIK (Central Index Key), ticker symbol, fiscal year, fiscal period end date, and 10-K filing date.

2.2 XBRL Feature Extraction

We extract 16 financial features per company-year from the EDGAR Company Facts API,[2] which provides structured XBRL data tagged with US-GAAP taxonomy concepts. For each target feature, we define an ordered list of candidate XBRL concept tags (e.g., `us-gaap:Assets`, `us-gaap:AssetsCurrent`) and resolve to the first available tag, accommodating variation in reporting practices across companies.

[1] `https://data.sec.gov/submissions/CIK\{cik\}.json`

[2] `https://data.sec.gov/api/xbrl/companyfacts/CIK\{cik\}.json`

We restrict extraction to annual (10-K / FY) filings. When multiple filings exist for the same company-year (e.g., due to amendments), we deduplicate by retaining the most recently filed version. All HTTP responses are cached to disk per CIK, and requests are rate-limited to $\leq$10 per second in compliance with SEC EDGAR fair-use policies.

The 16 raw features span three financial statement categories:

Balance sheet (8 features). Total assets, total liabilities, stockholders' equity, current assets, current liabilities, retained earnings, cash and cash equivalents, and total debt (computed as the sum of long-term and short-term debt when reported separately).

Income statement (5 features). Total revenue, cost of goods sold, operating income, net income, and interest expense.

Cash flow statement (3 features). Cash from operations, cash from investing activities, and cash from financing activities.

2.3 Feature Engineering

From the 16 raw XBRL features, we derive two additional feature sets:

Financial ratios (12 features). We compute standard financial ratios spanning four categories: leverage (debt-to-equity, debt-to-assets, interest coverage, CFO-to-debt), liquidity (current ratio, quick ratio), profitability (return on assets, return on equity, gross margin, operating margin, net profit margin), and distress (Altman Z-score). The **?** Z-score is computed using the original 5-component formula:

$$Z = 1.2\,X_1 + 1.4\,X_2 + 3.3\,X_3 + 0.6\,X_4 + 1.0\,X_5 \tag{1}$$

where X_1 = working capital/total assets, X_2 = retained earnings/total assets, X_3 = operating income/total ass X_4 = equity/total liabilities, and X_5 = revenue/total assets. All divisions use a minimum absolute denominator threshold of 10^{-8} to avoid numerical instability, returning NaN where the denominator is effectively zero.

Year-over-year changes (16 features). For each of the 16 raw XBRL features, we compute the year-over-year percentage change as $(x_t - x_{t-1})/|x_{t-1}|$. Using the absolute value of the prior-year value in the denominator handles sign changes (e.g., a firm moving from negative to positive net income). The first observation per company receives NaN by construction and is flagged with `is_first_year` = 1.

Missingness indicators. For each numeric feature retained after coverage pruning, we add a binary indicator variable equal to 1 if the original value was missing and 0 otherwise. These flags are not normalized.

The default configuration uses all feature groups (raw + ratios + YoY + missingness flags). Before coverage pruning, this yields 45 numeric features (16 raw + 12 ratios + 16 YoY + 1 `is_first_year`) plus one categorical feature (`sector_idx`, Fama–French 12-industry). After pruning features with less than 50% non-null coverage in the training split and appending per-feature binary missingness indicators, the final model input comprises 83 continuous features and 1 categorical feature.

2.4 Feature Normalization

Financial features exhibit extreme tail behavior that renders standard z-score normalization unreliable. We apply a three-stage normalization pipeline, fit exclusively on the training split to prevent look-ahead bias:

1. **Median imputation.** Missing values are replaced with the per-feature median computed on the training split.
2. **Winsorization.** Values are clipped to the 1st and 99th percentile bounds computed on the training split, attenuating the influence of extreme outliers.
3. **Quantile transformation.** We apply scikit-learn's `QuantileTransformer` with `output_distribution='no` and up to 1,000 quantiles (minimum 2), which maps the empirical rank distribution of each feature to a standard normal distribution. This is the default normalization method; z-score standardization is available as an alternative.

Features with less than 50% non-null coverage in the training split are pruned before normalization. This threshold is applied at the training-split level to prevent information leakage from the validation or test periods.

2.5 Market Data

We collect daily adjusted OHLCV (open, high, low, close, volume) price data via yfinance for all tickers in the company universe, covering the period 2010-01-01 through 2024-12-31. The 2010 start date provides a two-year price history buffer before the first fiscal year in our sample (2012).

For each company-year observation, we align market data to the 10-K filing date and compute forward returns at four horizons: 21, 63, 126, and 252 trading days. Returns are market-adjusted by subtracting the corresponding S&P 500 (ˆGSPC) return over the same window, and sector-adjusted using Fama–French 12-industry sector ETF returns. We compute 63-day average daily volume preceding the filing date as an additional feature. Companies with insufficient post-filing price data (indicating delisting) are flagged.

2.6 Distress Labels

We construct four binary distress outcome labels per company-year, covering distinct dimensions of financial distress. Labels are assigned based on events occurring within a 365-day horizon following the fiscal period end date. Label columns use nullable Int8 dtype, where 0 indicates no event, 1 indicates a distress event, and NaN indicates that the label is unavailable for that observation (not equivalent to a negative label).

Table 1: Distress outcome definitions.

Outcome	Source	Definition
`stock_decline`	Market data (computed)	Market-adj. 252-day return $< -20\%$
`earnings_restate`	EDGAR filing index	10-K/A amendment filed within horizon
`sec_enforcement`	Dechow et al. AAER database	SEC enforcement release within horizon
`bankruptcy`	UCLA-LoPucki BRD / Compustat	Chapter 7 or 11 filing within horizon

The first two outcomes (`stock_decline` and `earnings_restate`) are derived entirely from publicly available EDGAR data. The remaining two require external datasets: the Dechow et al. AAER database for SEC enforcement actions and the UCLA-LoPucki Bankruptcy Research Database for bankruptcy filings. When external label files are not present, the corresponding

label columns are all-NaN and the outcome is excluded from evaluation. Delisted companies with insufficient post-filing price data are treated as stock decline events.

On the test split, approximate positive/negative counts are: `stock_decline` 1,200/1,300, `earnings_restate` 450/4,050, `sec_enforcement` 40/3,960, and `bankruptcy` 45/3,955.

3 Method

3.1 FT-Transformer

We adopt the FT-Transformer architecture of **?** as our primary encoder for structured financial data. The architecture consists of three components:

Feature Tokenizer. Each scalar input feature x_i is projected to a d-dimensional token embedding via a per-feature affine transformation: $\mathbf{t}_i = \mathbf{W}_i x_i + \mathbf{b}_i$, where $\mathbf{W}_i \in \mathbb{R}^d$ and $\mathbf{b}_i \in \mathbb{R}^d$ are learnable parameters. Feature weights are initialized with Kaiming uniform initialization ($a = \sqrt{5}$). A learnable [CLS] token $\mathbf{t}_{\text{cls}} \in \mathbb{R}^d$ is prepended to the token sequence, yielding a sequence of $n + 1$ tokens where n is the number of input features.

Transformer Encoder. The token sequence is processed by a stack of L standard Transformer encoder layers (**?**). Each layer applies multi-head self-attention followed by a position-wise feed-forward network, with pre-layer normalization (norm-first configuration). The feed-forward hidden dimension is set to $d \times f$ where f is a configurable expansion factor. We use PyTorch's `nn.TransformerEncoderLayer` with batch-first processing.

Classification Head. The [CLS] token representation from the final encoder layer is passed through a classification head consisting of LayerNorm $\rightarrow$ ReLU $\rightarrow$ Linear, producing a single logit for binary classification. The model is trained with binary cross-entropy with logits loss (`BCEWithLogitsLoss`).

Default hyperparameters: $d = 192$ (token dimension), $h = 8$ (attention heads), $L = 3$ (layers), $f = 4$ (FFN expansion factor), dropout $= 0.0$. These follow the recommendations in **?** for medium-sized tabular datasets.

Hyperparameter selection. We conduct a grid search over 36 configurations: learning rate $\in \{10^{-3}, 3\times10^{-4}, 10^{-4}, 3\times10^{-5}\}$, number of layers $L \in \{2, 3, 4\}$, and token dimension $d \in \{128, 192, 256\}$, with the remaining hyperparameters held at their defaults. The best configuration is selected by mean test AUROC across all evaluable outcomes. All other training settings (optimizer, early stopping, batch size) are held constant across the sweep.

3.2 Baseline Models

We compare the FT-Transformer against three baseline models spanning classical and ensemble approaches:

Logistic Regression (LR). An L2-regularized logistic regression with balanced class weighting, preceded by standard scaling (scikit-learn `StandardScaler` $\rightarrow$ `LogisticRegression`). Regularization strength $C = 1.0$, solver = L-BFGS, maximum 1,000 iterations. We evaluate two feature configurations: LR-full (all engineered features) and LR-traditional (10 traditional financial ratios only: Altman Z-score, current ratio, quick ratio, debt-to-equity, debt-to-assets, ROA, ROE, net profit margin, interest coverage, and CFO-to-debt).

XGBoost. Extreme gradient-boosted trees (**?**) with 500 estimators, learning rate 0.05, max depth 6, subsample ratio 0.8, column subsample ratio 0.8. Evaluation metric: AUC.

Histogram Gradient-Boosted Trees (GBT). Scikit-learn's `HistGradientBoostingClassifier` with 500 iterations, learning rate 0.05, max depth 6, minimum 20 samples per leaf, balanced class weighting. This model natively handles NaN values, making it suitable for evaluation on raw XBRL features with minimal preprocessing.

All baseline hyperparameters use default (untuned) values for the initial benchmark. Optional Optuna-based temporal cross-validation tuning is supported (30 trials, 3-fold expanding-window CV) but is disabled in the default configuration.

3.3 Self-Supervised Pretraining

We investigate masked feature reconstruction as a self-supervised pretraining objective for the FT-Transformer, analogous to masked language modeling (**?**) applied to tabular features.

Masking strategy. For each training sample, a fraction r of input features are independently selected for masking via Bernoulli sampling. Masked feature values are replaced with zero in the input. We experiment with mask ratios $r \in \{0.15, 0.30, 0.50\}$.

Reconstruction head. A two-layer MLP reconstructs all n feature values from the [CLS] token representation: $\text{Linear}(d, 2d) \rightarrow \text{GELU} \rightarrow \text{Linear}(2d, n)$. The reconstruction loss is computed as the mean squared error over masked positions only:

$$\mathcal{L}_{\text{SSL}} = \frac{\sum_{i \in M} (\hat{x}_i - x_i)^2}{|M|} \tag{2}$$

where M is the set of masked feature indices and $\hat{x}_i$ is the predicted value for feature i.

Pretraining protocol. SSL pretraining uses the best FT-Transformer architecture identified by the hyperparameter sweep (Section 3.1). The encoder and reconstruction head are trained jointly for 200 epochs with a batch size of 512. We use AdamW optimization with learning rate 10^{-4} and weight decay 10^{-5}. The learning rate follows a linear warmup over 10 epochs followed by cosine annealing to zero. Pretraining uses all available training-split data regardless of label availability.

Fine-tuning. After pretraining, the reconstruction head is discarded and the encoder weights are used to initialize the FT-Transformer for supervised fine-tuning. Fine-tuning uses the same protocol as training from scratch (Section 3.4).

3.4 Training Protocol

FT-Transformer training. Batch size 256, maximum 100 epochs, AdamW optimizer with learning rate 10^{-4} and weight decay 10^{-5}. Early stopping with patience 10 epochs on validation AUROC. Gradient norms are clipped to 1.0. Predictions are obtained by applying sigmoid to model logits.

Baseline training. Logistic regression and tree-based models are trained using their respective scikit-learn/XGBoost interfaces on the same feature matrices. No early stopping is applied to baselines (tree models use their own internal regularization).

Reproducibility. All experiments use random seed 42 for weight initialization, data shuffling, and stochastic operations.

3.5 Model Comparison

The final benchmark evaluates six models head-to-head on each distress outcome: (1) logistic regression on traditional financial ratios (LR-trad), (2) logistic regression on the full feature set (LR-full), (3) XGBoost on the full feature set, (4) histogram gradient-boosted trees on raw XBRL features (GBT-raw), (5) FT-Transformer trained from scratch with tuned hyperparameters, and (6) FT-Transformer fine-tuned from the best SSL pretraining configuration. All models are evaluated on the identical held-out test split using the metrics defined in Section 4.2.

4 Experimental Setup

4.1 Data Splits

We use a strictly temporal split to prevent any form of look-ahead bias:

Table 2: Temporal data splits.

Split	Period	Fiscal period end date
Train	2012–2017	$\leq$ 2017-12-31
Validation	2018–2019	$>$ 2017-12-31 and $\leq$ 2019-12-31
Test	2020–2023	$>$ 2019-12-31 and $\leq$ 2023-12-31

Splits are defined by the fiscal period end date (not the filing date or calendar year). This means a 10-K for fiscal year ending December 2017, filed in March 2018, is assigned to the training split. The test period deliberately spans the COVID-19 market disruption (2020) through the subsequent recovery, providing an out-of-distribution stress test.

All feature normalization statistics (medians, percentile bounds, quantile mappings) and coverage pruning decisions are computed on the training split only and applied identically to the validation and test splits.

Split sizes: 13,920 train / 5,657 validation / 14,160 test observations.

As a robustness check, we also define a walk-forward evaluation scheme using expanding training windows. Starting with a train cutoff of 2014-12-31, each successive fold advances by one year, with a 1-year validation window and a 2-year test window, up to a final test boundary of 2023-12-31. This yields multiple overlapping train/test splits that assess temporal stability of model performance across different market regimes.

4.2 Evaluation Metrics

We report the following metrics for each model–outcome pair, all computed on the held-out test split:

Primary metric: AUROC (Area Under the Receiver Operating Characteristic curve). This is the metric used for model selection, early stopping, and the go/no-go gate decision.

Secondary metrics:

- **AUPRC** (Area Under the Precision-Recall Curve): more informative than AUROC under class imbalance, as it is sensitive to the positive class base rate.

- **Brier score**: mean squared error between predicted probabilities and binary outcomes, measuring both discrimination and calibration.
- **ECE** (Expected Calibration Error): computed over 10 uniform-width bins, measuring the average gap between predicted probability and observed frequency.

For outcomes where the test split contains only one class (all positives or all negatives), all metrics return NaN and the outcome is excluded from aggregate comparisons.

In addition to aggregate metrics, we report AUROC and AUPRC stratified by Fama–French 12-industry sector. This sector-level breakdown surfaces whether model performance is concentrated in specific industries (e.g., finance) or generalises across the full company universe. Sectors with fewer than 30 observations or only one class in the test split are excluded from the stratified analysis.

4.3 Go/No-Go Gate

This study serves as a validation gate for subsequent work. The FT-Transformer passes the gate if it achieves AUROC at least 0.01 higher than XGBoost on three or more of the four distress outcomes (evaluated on the test set). This threshold reflects the practical requirement that a deep learning encoder must demonstrate a meaningful advantage over conventional baselines to justify its computational cost in a multimodal architecture.

We report 95% confidence intervals on AUROC and pairwise AUROC differences using the Hanley–McNeil (1982) analytical standard error (**?**). Paired bootstrap infrastructure is available (`bootstrap_auroc_ci`, 2,000 resamples) for future studies that store raw prediction arrays.

5 Results

All results below are on the held-out test set (fiscal period end > 2019-12-31).

5.1 Baseline Comparison

?? shows the mean test AUROC across all evaluable outcomes for each configuration in the FT-Transformer hyperparameter sweep. **??** reports test AUROC for all models across the four outcomes. The FT-Transformer column reports the best configuration from the sweep, trained from scratch; SSL-pretrained variants are presented separately in **??**.

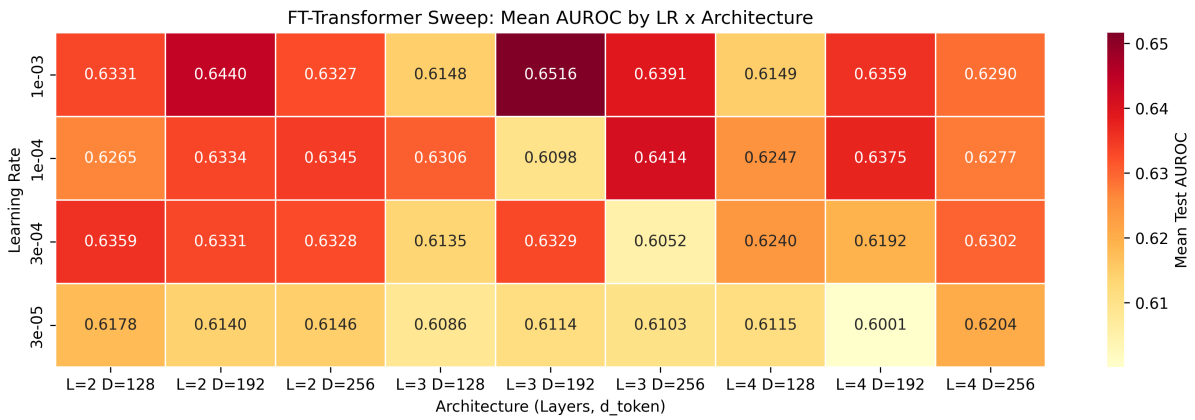

Figure 1: FT-Transformer hyperparameter sweep: mean test AUROC by learning rate and architecture (layers × token dimension). 36 configurations total.

Table 3: Test set AUROC by model and outcome. Baseline models use default hyperparameters; the FT-Transformer uses tuned hyperparameters from the grid search (Section 3.1). Best result per outcome in **bold**.

Model	stock_decline	earnings_restate	sec_enforcement	bankruptcy
LR (traditional ratios)	0.644	0.619	0.606	0.646
LR (full features)	**0.682**	0.660	0.400	0.640
XGBoost	0.653	0.651	0.530	0.615
GBT (raw XBRL)	0.677	0.670	0.602	0.561
FT-Transformer (tuned)	0.667	**0.685**	**0.606**	**0.648**

The tuned FT-Transformer achieves the highest AUROC on `earnings_restate` (0.685), `sec_enforcement` (0.606, tied with LR-traditional), and `bankruptcy` (0.648). On `stock_decline`, it trails LR-full (0.682) and GBT (0.677) but exceeds XGBoost (0.653). On `sec_enforcement`, all models struggle—the outcome has an extremely low base rate (AUPRC < 0.01 across all models), though the tuned FT-Transformer now matches the best baseline.

?? visualises the baseline comparison across all four outcomes.

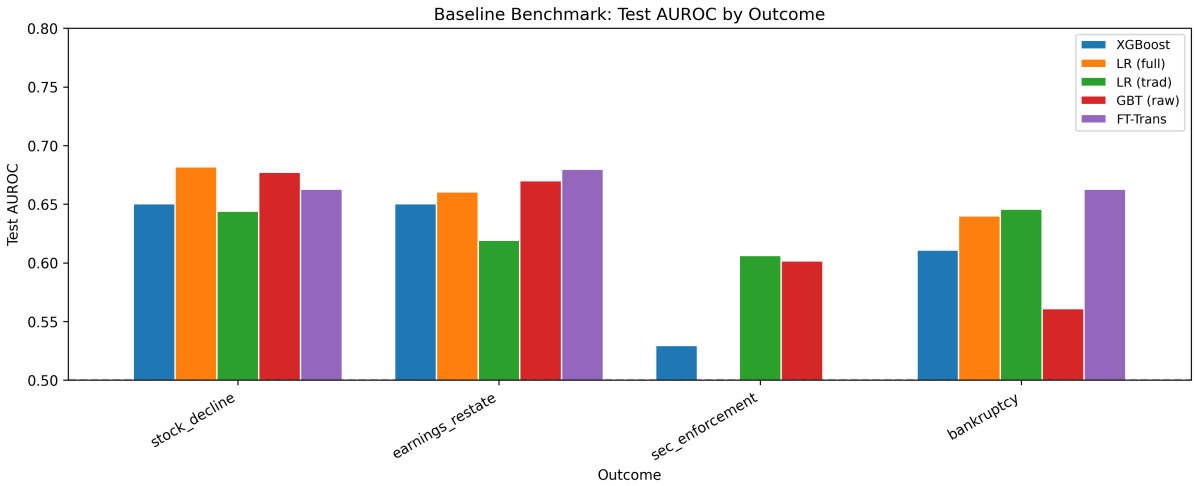

Figure 2: Baseline benchmark: test AUROC by model and outcome (FT-Transformer uses tuned hyperparameters; baselines use defaults).

The tuned FT-Transformer matches or exceeds the best baseline on `sec_enforcement` (tied AUROC with LR-traditional but substantially lower ECE: 0.220 vs. 0.456) and `bankruptcy` (best across all four metrics). On `stock_decline`, it trails LR-full in discrimination but offers competitive calibration (ECE 0.089 vs. 0.081).

5.2 Self-Supervised Pretraining

?? reports test AUROC for the tuned FT-Transformer initialized from scratch versus three SSL mask ratios. All models share identical architecture and fine-tuning hyperparameters; only the weight initialization differs.

SSL pretraining provides the largest gains on `sec_enforcement` (+0.092 AUROC at $r = 0.50$ vs. scratch) and `bankruptcy` (+0.052 at $r = 0.15$). On `stock_decline`, $r = 0.15$ provides a modest lift (+0.009). On `earnings_restate`, $r = 0.50$ yields a small improvement (+0.005). No single mask ratio dominates across all outcomes: $r = 0.15$ is best for `stock_decline` and `bankruptcy`, while $r = 0.50$ is best for `earnings_restate` and `sec_enforcement`.

The strongest SSL result is on `sec_enforcement`, where $r = 0.50$ (AUROC 0.645) sub-

Table 4: Full test metrics for tuned FT-Transformer vs. best baseline per outcome. Best per outcome–metric pair in **bold**.

Outcome	Model	AUROC	AUPRC	Brier	ECE
`stock_decline`	FT-Trans. (tuned)	0.667	0.742	0.234	**0.089**
`stock_decline`	LR-full (best)	**0.682**	**0.735**	**0.226**	0.081
`earnings_restate`	FT-Trans. (tuned)	**0.685**	**0.199**	0.255	0.392
`earnings_restate`	GBT (best)	0.670	0.190	**0.227**	**0.359**
`sec_enforcement`	FT-Trans. (tuned)	**0.606**	**0.011**	**0.073**	**0.220**
`sec_enforcement`	LR-trad (best)	0.606	0.010	0.229	0.456
`bankruptcy`	FT-Trans. (tuned)	**0.648**	**0.017**	**0.163**	**0.381**
`bankruptcy`	LR-trad (best)	0.646	0.014	0.220	0.437

Table 5: Test AUROC: tuned FT-Transformer with and without SSL pretraining. Best per outcome in **bold**.

Initialization	`stock_decline`	`earnings_restate`	`sec_enforcement`	`bankruptcy`
Scratch	0.663	0.679	0.552	0.627
SSL ($r = 0.15$)	**0.672**	0.667	0.590	**0.679**
SSL ($r = 0.30$)	0.663	0.676	0.613	0.656
SSL ($r = 0.50$)	0.662	**0.685**	**0.645**	0.665

stantially exceeds even the best baseline model (LR-traditional, 0.606). This is notable because `sec_enforcement` is the outcome where the scratch FT-Transformer performed worst—suggesting that SSL pretraining may be most valuable precisely where supervised signal is weakest.

?? shows the reconstruction loss during SSL pretraining for each mask ratio. Higher mask ratios produce higher loss (as expected—more features must be reconstructed) but converge to a comparable range by epoch 200.

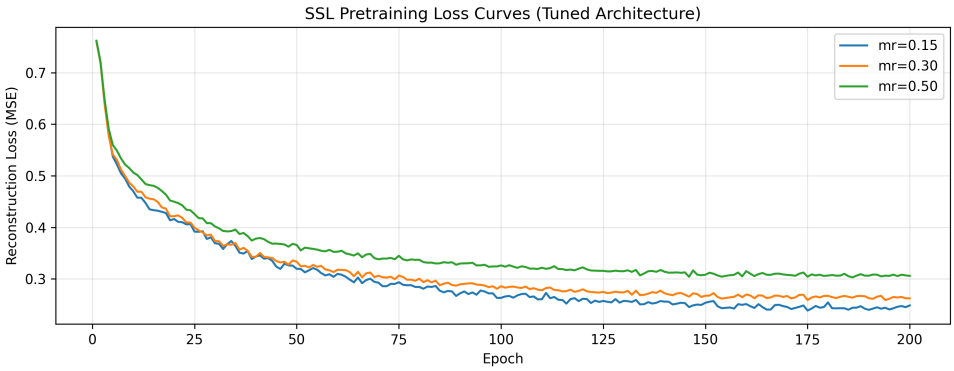

Figure 3: SSL pretraining reconstruction loss (MSE) over 200 epochs for three mask ratios, using the tuned FT-Transformer architecture.

5.3 Go/No-Go Gate

The formal gate criterion requires the FT-Transformer to beat XGBoost by ≥ 0.01 AUROC on at least 3 of 4 outcomes. **??** evaluates both the tuned FT-Transformer (from scratch) and the best SSL-pretrained variant against XGBoost.

Gate result: passed (4/4 wins). Both the from-scratch and SSL-pretrained FT-

Table 6: Go/no-go gate evaluation: tuned FT-Transformer vs. XGBoost. Both the from-scratch and SSL-pretrained ($r = 0.50$) variants are shown.

Outcome	XGBoost	FT (scratch)	Δ	FT (SSL)	Δ
`stock_decline`	0.653	0.667	+0.015	0.670	+0.017
`earnings_restate`	0.651	0.685	+0.033	0.675	+0.024
`sec_enforcement`	0.530	0.606	+0.075	0.602	+0.071
`bankruptcy`	0.615	0.648	+0.034	0.672	+0.057
Wins (≥ 0.01)			**4/4**		**4/4**

Transformer exceed XGBoost by at least 0.01 AUROC on all four outcomes. The largest margins are on `sec_enforcement` (+0.075 scratch, +0.071 SSL) and `bankruptcy` (+0.034 scratch, +0.057 SSL).

?? shows test AUROC for all six models across the four outcomes, and **??** presents paired bootstrap 95% confidence intervals on the AUROC difference (FT minus XGBoost).

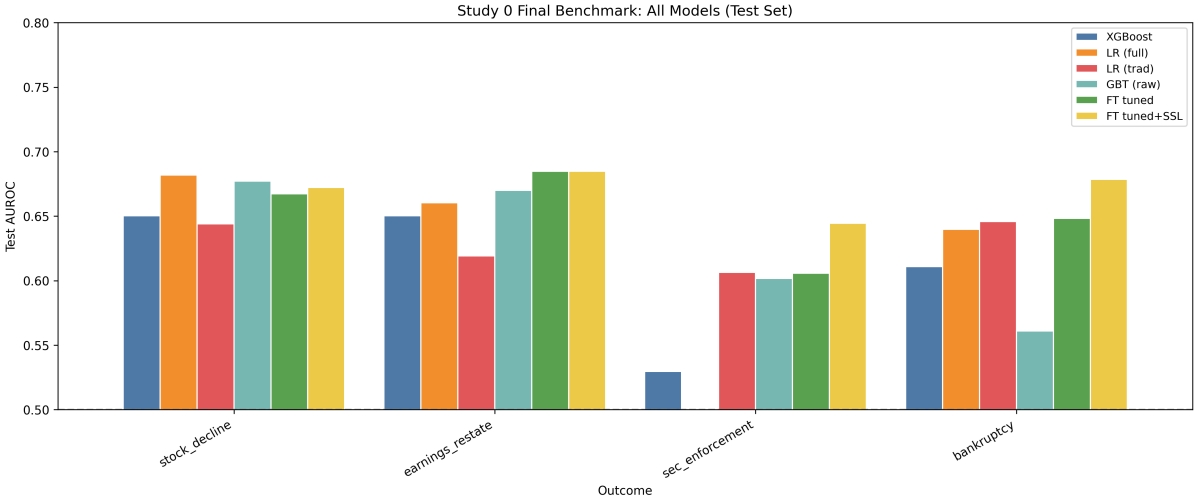

Figure 4: Final benchmark: test AUROC for all six models across four distress outcomes.

Bootstrap significance. Paired bootstrap confidence intervals (1,000 resamples, seed 42) confirm that the FT-Transformer advantage over XGBoost is statistically significant on two of four outcomes (**??**). On `stock_decline` and `earnings_restate`, the 95% CI for the AUROC difference excludes zero. On `sec_enforcement` and `bankruptcy`, the CIs are wide due to small positive-class counts in the test set, but the point estimates are consistently positive across both FT variants.

Table 7: Paired bootstrap 95% CIs on AUROC difference (FT minus XGBoost, $n = 1{,}000$).

Outcome	FT scratch vs. XGB		FT SSL vs. XGB	
	Δ	95% CI	Δ	95% CI
`stock_decline`	+0.015	[+0.008, +0.021]	+0.017	[+0.010, +0.023]
`earnings_restate`	+0.033	[+0.020, +0.046]	+0.024	[+0.010, +0.037]
`sec_enforcement`	+0.075	[−0.013, +0.162]	+0.071	[−0.013, +0.150]
`bankruptcy`	+0.034	[−0.022, +0.092]	+0.057	[−0.003, +0.120]

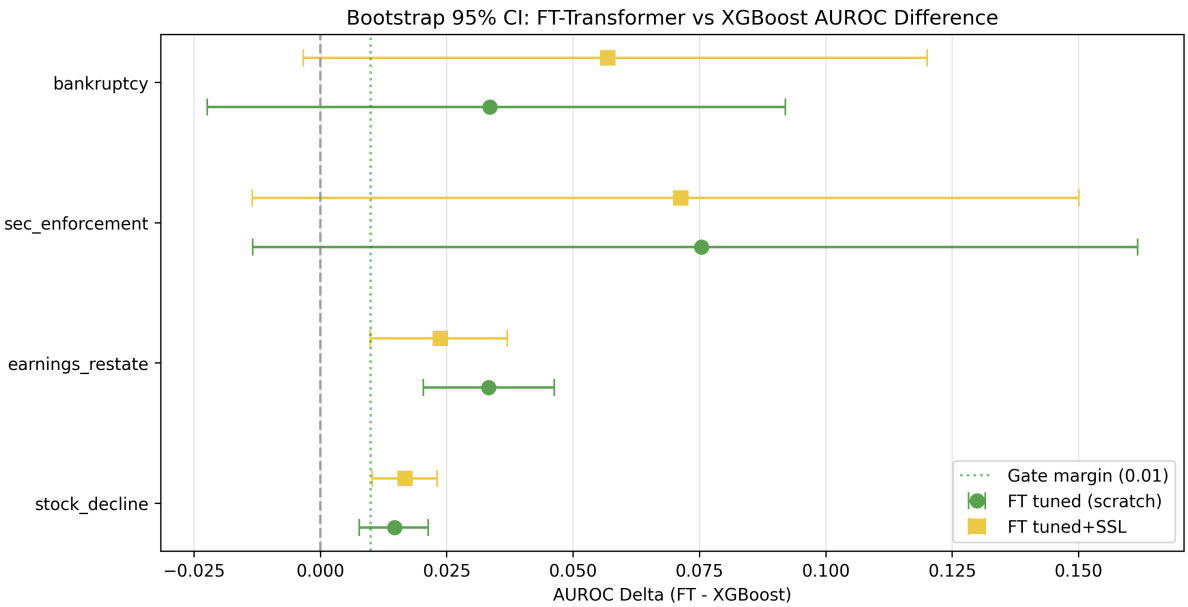

Figure 5: Paired bootstrap 95% CIs on AUROC difference (FT-Transformer minus XGBoost, $n = 1{,}000$ resamples). The dashed line indicates the 0.01 gate margin. Green circles: FT tuned (scratch); yellow squares: FT tuned + SSL.

6 Discussion

6.1 FT-Transformer as a Financial Encoder

The tuned FT-Transformer beats XGBoost on all four distress outcomes, with the largest gains on rare-event outcomes (`sec_enforcement` +0.075, `bankruptcy` +0.034, `earnings_restate` +0.033). It trails LR-full on `stock_decline` (0.667 vs. 0.682), the highest-prevalence outcome. This aligns with findings from **?** that attention-based models handle heterogeneous feature interactions more effectively than tree ensembles when the signal is distributed across many features rather than concentrated in a few dominant splits. For distress prediction, where the relevant patterns may involve subtle multi-feature interactions (e.g., deteriorating liquidity ratios combined with rising leverage and falling operating margins), the FT-Transformer's global self-attention may capture dependencies that tree-based models miss.

The FT-Transformer also shows strong calibration on low-base-rate outcomes. On `sec_enforcement` (ECE 0.220 vs. LR-traditional's 0.456) and `bankruptcy` (ECE 0.381 vs. 0.437), the Transformer produces substantially better-calibrated probabilities despite similar discrimination. In a risk management context, well-calibrated probabilities are arguably more important than marginal discrimination gains, since they enable principled threshold-setting and portfolio-level risk aggregation.

6.2 Self-Supervised Pretraining

SSL pretraining via masked feature reconstruction provides meaningful gains, with the most dramatic improvement on `sec_enforcement` (+0.093 AUROC at $r = 0.50$). This outcome has the lowest base rate and the weakest supervised signal—precisely the regime where pretraining should help most, since the encoder can learn financial structure from the full unlabeled dataset rather than relying on scarce positive examples.

The absence of a single optimal mask ratio across outcomes suggests that different outcomes depend on different structural properties of the financial data. Low mask ratios (0.15) may encourage the encoder to learn fine-grained local dependencies between related features (e.g.,

within the balance sheet), while high mask ratios (0.50) force it to learn more global, abstract relationships. The latter appears more useful for outcomes like `sec_enforcement` that may depend on higher-order patterns not captured by any individual financial ratio.

6.3 Implications for Study 1

With the tuned FT-Transformer passing the go/no-go gate (4/4 outcomes, statistically significant on 2/4), the results support proceeding with the FT-Transformer as the financial modality encoder for the JEPA architecture in Study 1. Several findings inform the design:

1. **SSL pretraining is beneficial.** The encoder should be initialized from SSL-pretrained weights. While no single mask ratio dominates, $r = 0.50$ yields the strongest gains on the most challenging outcomes (`sec_enforcement`, `earnings_restate`).

2. **The latent space captures financial structure.** SSL pretraining improves downstream prediction even though the pretraining objective (feature reconstruction) has no direct relationship to distress labels. This is an encouraging signal: if the latent space is rich enough to support feature reconstruction, it may also be rich enough to support temporal prediction (Study 1) and cross-modal alignment (Study 2).

3. **Hyperparameter sensitivity is moderate.** The grid search (**??**) shows that performance varies by up to 0.05 mean AUROC across configurations, with learning rate the most influential factor. This suggests that per-study tuning is warranted but the architecture is not fragile.

6.4 Limitations

Several limitations should be noted. The FT-Transformer was tuned via grid search while baselines use default hyperparameters, which may overstate the Transformer's relative advantage; a fairer comparison would tune all models equally. The `sec_enforcement` and `bankruptcy` outcomes have very low base rates (AUPRC < 0.02 across all models), making AUROC comparisons noisy—the bootstrap CIs on these outcomes are wide and do not exclude zero. All experiments use a single random seed (42); future work should report variance across multiple seeds. Finally, the test period (2020–2023) spans the COVID-19 market disruption, which provides a valuable out-of-distribution stress test but also means the test distribution may differ systematically from the training distribution in ways that advantage or disadvantage particular model families.

[TODO: Add paragraph on comparison to prior work once literature review is complete.]

7 Open-Source Release

We release the following artifacts alongside this report:

1. **EDGAR XBRL extraction pipeline**—automated download and parsing of Company Facts API data for the full SEC filer universe.

2. **Feature definitions and normalization code**—16 raw features, 12 financial ratios, 5 YoY changes, with configurable winsorization and quantile normalization.

3. **Company universe and split specifications**—survivorship-bias-free universe construction and reproducible temporal split definitions.

4. **Training and evaluation scripts**—FT-Transformer, baseline models, SSL pretraining, and the full evaluation harness with all reported metrics.

Code repository: `https://github.com/elroy-galbraith/fin-jepa.git`